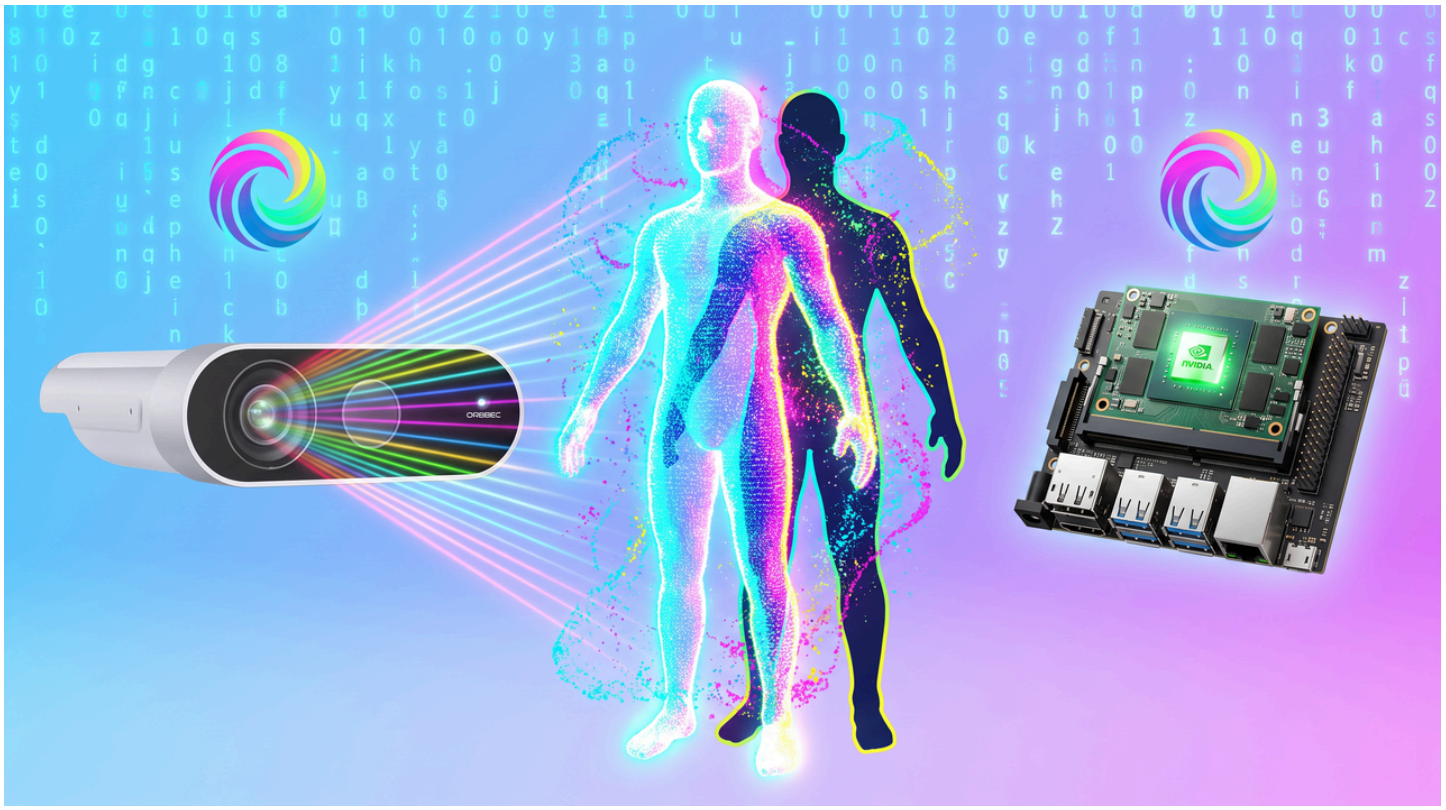
Jetson Orin Nano + Ubuntu 22.04 + Orbbec Femto Mega 人物抠取项目复现手册 🎬



> 这是一份可以直接贴到飞书文档里的复现记录。目标不是“看起来能跑”，而是把项目从环境、相机、模型、Web 页面到常见坑都串起来，让读者能一步一步复现出人物抠取效果。
>
> 🔗 GitHub

1. 项目是做什么的？🧩

这个项目的目标是做一套“人物图像扣取 / 视频人像抠图 / 背景替换”系统。

它的核心思路是：

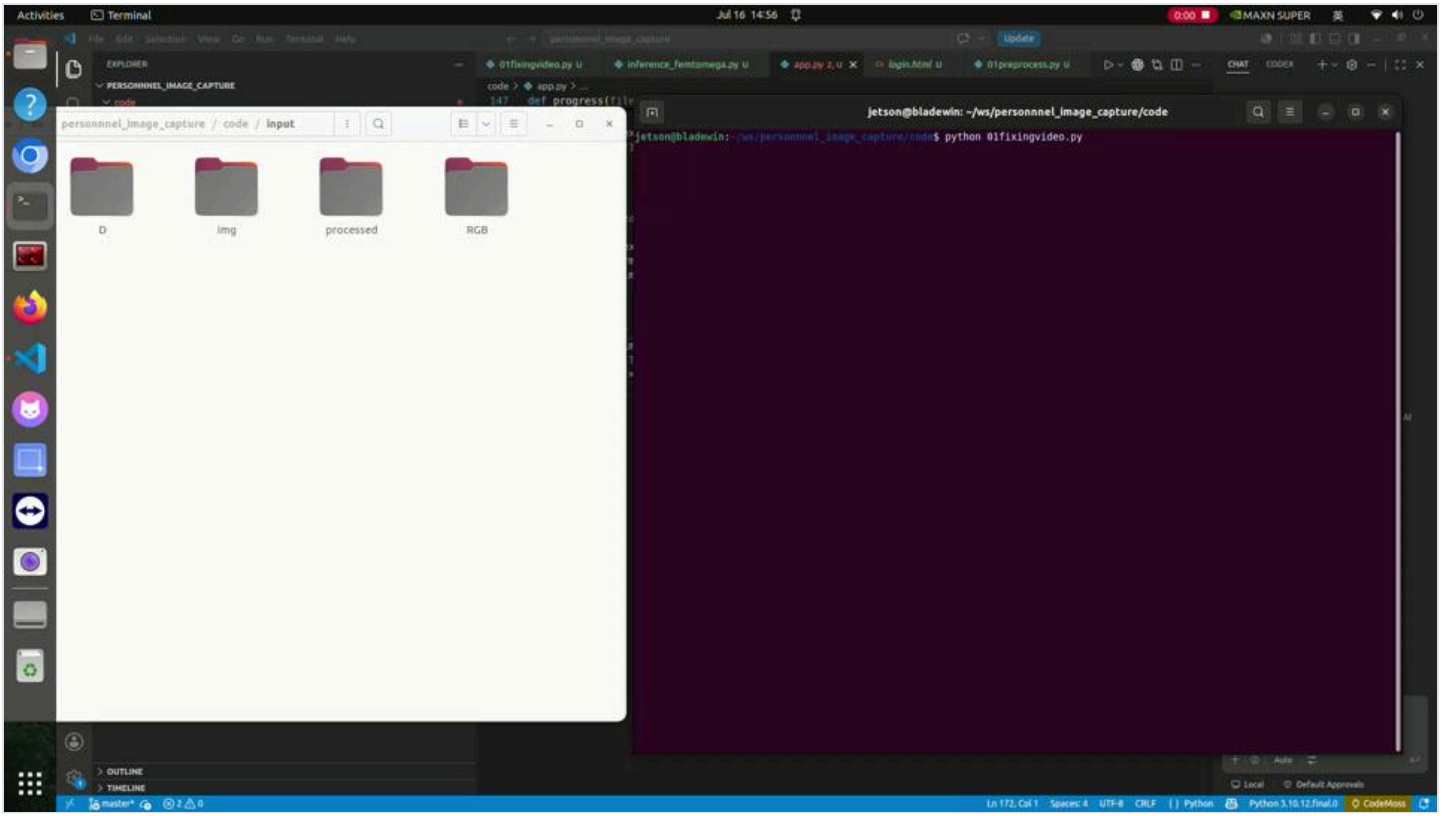
1. 使用 Orbbec 深度相机采集 RGB 视频和 Depth 视频。
2. 根据深度信息先做一次预处理，把远处背景干扰弱化。

3. 使用 Robust Video Matting 风格的神经网络模型输出：
    - 前景 `foreground`
    - 透明度遮罩 `alpha`
    - 合成后视频 `composition`
4. 通过 Flask Web 页面上传视频、触发推理、查看和下载结果。

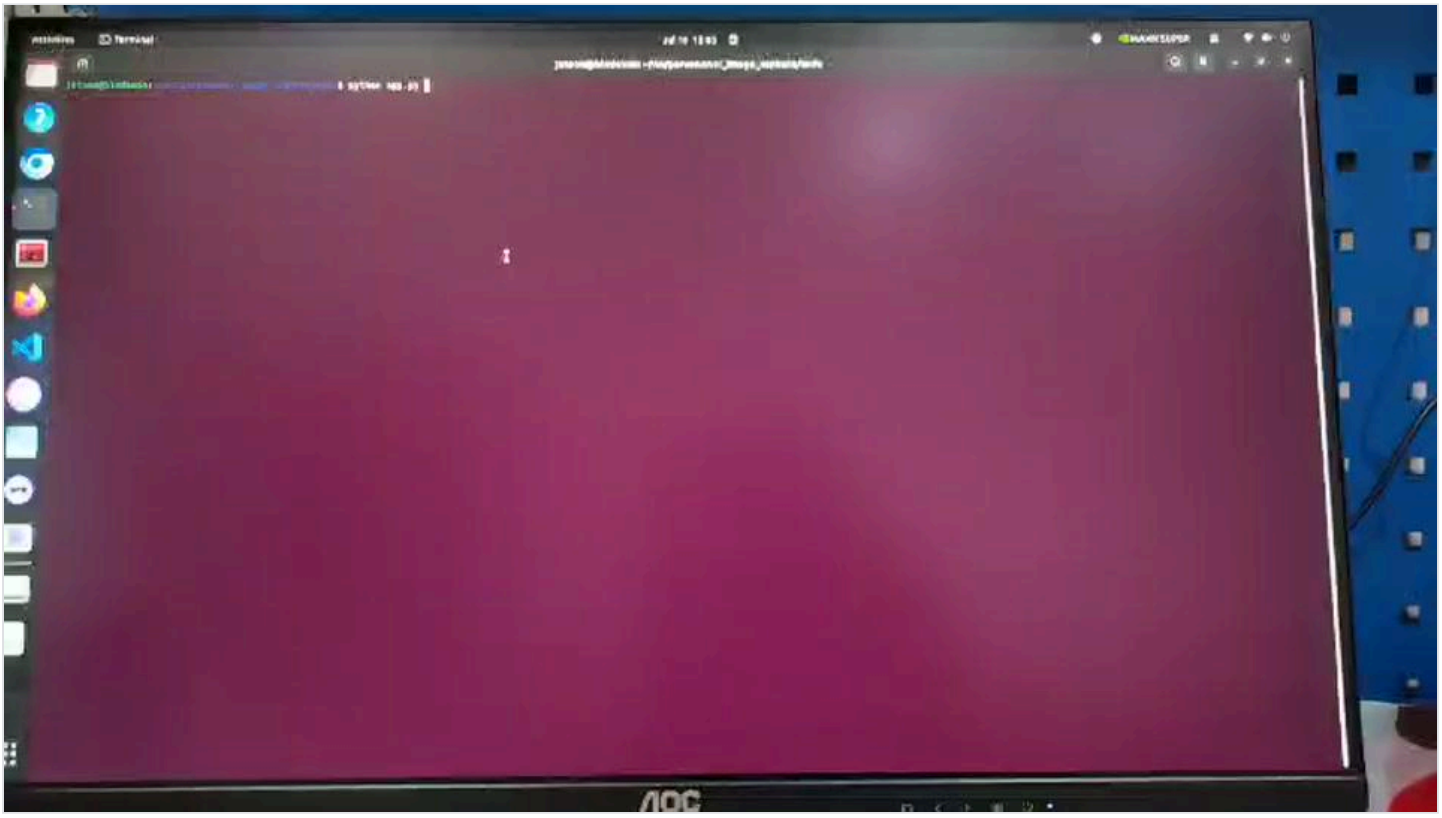
本次复现平台：

项目	配置
开发板	Jetson Orin Nano
系统	Ubuntu 22.04
相机	Orbbec Femto Mega
后端	Flask + Python
推理框架	PyTorch + CUDA
模型	RVM / `rvm_mobilenetv3.pth`
主要用途	人物视频抠取、alpha 生成、背景替换测试

本地代码运行：



Web端演示：



2. 先说结论：原项目不能直接一键跑 😅

原始项目链路是完整的，但工程上有一些小缺口：

- `app.py` 默认要求登录，不适合快速测试 Web 推理。
- `charts.html` 里的按钮原来主要是模拟进度条，没有真正调用 `/upload` 和 `/process`。
- `app.py` 里有未导入的 `uuid`、`shutil`。
- `/process` 原来调用了不存在的 `remove(input, session=session)`。
- `inference.py` 里虽然写了核心推理函数，但命令行入口不完整。
- `invoking3.py` 想用 CLI 调 `inference.py`，但两者没有真正接上。
- 项目里缺少 `rvm_mobilenetv3.pth` 权重文件。
- `01fixingvideo.py` 引用了 `utils.py`，但项目目录里没有，需要从 Orbbec 示例里补。
- 原项目面向 Astra+，这次是 Femto Mega，网络连接方式和 SDK 版本需要留心。

所以复现策略是：**先让核心链路跑通，再让 Web 端调用本地推理工具。**

3. 推荐目录结构 📁

迁移到 Jetson 后建议类似这样放：

代码块

```
~/ws/personnnel_image_capture/code/
├── app.py
├── inference.py
├── inference_femtomega.py
├── inference_utils.py
├── rvm_mobilenetv3.pth
├── utils.py
├── model/
├── templates/
│   ├── charts.html
│   ├── login.html
│   └── register.html
├── static/
├── input/
│   ├── RGB/
│   ├── D/
│   └── processed/
├── output/
└── uploads/
```

重点文件说明：

文件	作用
`01fixingvideo.py`	调 Orbbec SDK 采集 RGB + Depth 视频
`01preprocess.py`	根据深度阈值预处理 RGB 视频
`inference.py`	RVM 模型推理核心
`app.py`	Flask Web 后端
`templates/charts.html`	Web 测试页面
`rvm_mobilenetv3.pth`	必须手动放入的模型权重
`utils.py`	Orbbec 示例里的帧转换工具

4. Jetson 基础环境准备 🛠️

先确认系统和 JetPack：

代码块

```
cat /etc/nv_tegra_release
python3 --version
```

安装基础工具：

代码块

```
sudo apt update
sudo apt install -y \
  python3-venv python3-pip python3-dev \
```

```
    build-essential cmake git ffmpeg \
    libopenblas-dev libgl1 libglib2.0-0 \
    python3-opencv v4l-utils
```

创建虚拟环境：

代码块

```
python3 -m venv ~/venvs/person_matting --system-site-packages
source ~/venvs/person_matting/bin/activate
pip install --upgrade pip setuptools wheel
```

安装 Python 依赖：

代码块

```
pip install numpy pillow tqdm flask av pims werkzeug
```

> 小提醒 ☕：Jetson 上的 PyTorch 不建议直接 `pip install torch torchvision`，容易装到不带 CUDA 或不兼容的版本。建议按 JetPack 版本安装 NVIDIA 提供的 Jetson PyTorch wheel。

安装完成后检查：

代码块

```
python3 - <<'PY'
import torch
print(torch.__version__)
print("CUDA available:", torch.cuda.is_available())
PY
```

理想输出：

```
CUDA available: True
```

5. 安装 Orbbec SDK / pyorbbecsdk 📷

Femto Mega 推荐使用 Orbbec SDK v2 对应的 Python SDK。

可以先尝试：

```
pip install pyorbbecsdk2
```

检查能否导入：

```
python3 - <<'PY'
from pyorbbecsdk import *
print("pyorbbecsdk import ok")
PY
```

如果导入失败，需要确认：

- 当前 Python 版本是否被 SDK 支持。
- 当前架构是否为 `aarch64`。
- 是否使用了对应 Linux ARM64 的 wheel 或源码编译版本。

- 是否安装了 Orbbec SDK 的运行库和 udev rules。

如果是 USB 连接相机，可以检查设备：

```
代码块
1   lsusb
```

如果是 Femto Mega 网络连接，需要先确认网络：

```
代码块
1   ping <Femto-Mega-IP>
```

Femto Mega 走网络时，不能完全照搬普通 USB 的 `Pipeline()` 写法。通常需要参考 Orbbec 网络相机示例，用类似下面的方式创建网络设备：

```
代码块
1   ctx = Context()
2   device = ctx.create_net_device(ip, 8090)
3   pipeline = Pipeline(device)
```

6. 补齐项目缺失文件 🧱

6.1 补`utils.py`

项目的 `01fixingvideo.py` 有这一行：

```
代码块
```

```
from utils import frame_to_bgr_image
```

但原项目目录里没有 `utils.py`。

解决方式：

1. 从 Orbbec `pyorbbecsdk/examples/utils.py` 拷贝到当前代码目录。
2. 或者把 `frame_to_bgr_image` 的逻辑直接合并进 `01fixingvideo.py`。

不补的话，运行采集脚本会直接报：

代码块

```
ModuleNotFoundError: No module named 'utils'
```

6.2 补`rvm_mobilenetv3.pth`

推理脚本需要模型权重：

代码块

```
rvm_mobilenetv3.pth
```

放置位置：

代码块

```
app.py 同级目录
```

如果没有这个文件，Web 端会返回：

```
代码块
1   Missing rvm_mobilenetv3.pth. Put the checkpoint in the 3d vision source
    directory.
```

7. 先跑通命令行链路 🧪

不要一上来就跑 Web。建议先用命令行分三步验证。

7.1 采集 RGB + Depth

```
代码块
1   python3 01fixingvideo.py -m HW
```

如果硬件对齐模式不支持，换成：

```
代码块
1   python3 01fixingvideo.py -m SW
```

成功后应该生成：

```
代码块
1   input/RGB/rgb.mp4
```

```
2   input/D/depth.mp4
```

常见问题：

现象	可能原因	处理方式
找不到设备	udev rules 未装 / 相机未连接	重新安装 Orbbec 环境脚本，检查线缆
`utils` 导入失败	缺少 `utils.py`	从 Orbbec 示例复制
Depth 画面异常	深度格式或网络流处理不同	参考 Femto Mega 官方示例调整
网络相机无法打开	没用 `create_net_device`	改成网络设备 Pipeline

7.2 深度预处理

代码块

```
1   python3 01preprocess.py
```

成功后应该生成：

代码块

```
1   input/processed/rgb_processed.mp4
```

这里要注意一个坑：原代码里的阈值是：

代码块

```
depth_threshold = 100
```

它处理的是归一化后的 8-bit 深度图，不是真实毫米深度。

如果希望效果稳定，建议后续改成直接使用原始 `uint16` 深度值，比如保留 0.5m 到 2.5m 内的人物区域。

7.3 模型推理

不要直接跑 `invoking3.py`，它和 `inference.py` 的命令行接口没有真正接上。

代码块

```
python inference_femtomega.py
```

成功后会看到：

代码块

```
output/com/composition.mp4
output/alpha/alpha.mp4
output/for/foreground.mp4
```

8. Web 端改造说明 🌐

为了方便测试，我们对 Web 端做了几个关键调整。

8.1 取消登录

测试阶段不需要账号系统，所以把：

```
代码块
1  @app.route("/")
2  def hello():
3      return render_template("login.html")
```

改成直接进入测试页面：

```
代码块
1  @app.route("/")
2  def hello():
3      ensure_runtime_dirs()
4      return render_template("charts.html")
```

同时 `/index` 和 `/login` 也直接返回 `charts.html`。

这样浏览器访问：

```
代码块
1  http://<Jetson-IP>:5000
```

就能直接进入测试界面，不再卡在登录页。

8.2 修复上传接口

原始 `app.py` 少了：

代码块

```
import uuid
import shutil
```

并且允许的文件格式里没有 `mp4` 。

修复后支持：

代码块

```
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "gif", "avi", "mp4", "mov"}
VIDEO_EXTENSIONS = {"avi", "mp4", "mov"}
```

8.3 修复`/process`

原始 `/process` 里有：

代码块

```
output = remove(input, session=session)
```

但 `remove` 和 `session` 都没有定义。

修复方式：在 `/process/<filename>` 中懒加载 `torch` 和 `Converter` ，然后调用本地 RVM 推理：

代码块

```
from inference import Converter
```

```
2
3   device = "cuda" if torch.cuda.is_available() else "cpu"
4   converter = Converter("mobilenetv3", CHECKPOINT_PATH, device)
5   converter.convert(...)
```

8.4 修复前端按钮

原始 `charts.html` 的按钮很多只是：

```
代码块
1   simulateProgress(...)
```

也就是“假进度条”，不会真的上传或推理。

修复后：

按钮	实际调用
Upload RGB Video	`POST /upload`
Generate Result	`GET /process/<filename>`
Download Result	`GET /download/<filename>`
Clear	`GET /clear`

9. 启动 Web 测试 🚀

进入代码目录：

代码块

```
cd ~/ws/personnnel_image_capture/code
source ~/venvs/person_matting/bin/activate
python3 app.py
```

看到类似输出：

代码块

```
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.110.83:5000
```

浏览器打开：

代码块

```
http://192.168.110.83:5000
```

测试流程：

1. 点击 `Upload RGB Video` 上传视频。
2. 上传成功后， `Generate Result` 按钮会变成可点击。
3. 点击 `Generate Result` 开始本地模型推理。
4. 推理完成后页面会显示结果视频。
5. 点击 `Download Result` 下载合成后视频。

终端里应该看到：

```
POST /upload HTTP/1.1" 200 -
GET /process/xxx.mp4 HTTP/1.1" 200 -
GET /download/xxx.mp4 HTTP/1.1" 200 -
```

如果只看到：

代码块

```
POST /index HTTP/1.1" 200 -
```

说明页面还在旧版本，或者浏览器缓存没有刷新。

解决方式：

- 重启 Flask。
- 浏览器强制刷新。
- 确认 Jetson 上的 `templates/charts.html` 已经替换成新版。

10. 复现过程中最容易踩的坑 🕳️

坑 1：Flask 页面能打开，但没有真正上传

现象：

代码块

```
GET / HTTP/1.1" 200 -
POST /index HTTP/1.1" 200 -
```

但没有：

代码块

```
POST /upload
GET /process
```

原因：旧版 `charts.html` 的按钮只模拟进度，没有真正绑定上传和推理接口。

处理：替换新版 `charts.html`。

坑 2：浏览器强制要求登录

原因：`/` 默认返回 `login.html`。

处理：测试阶段把 `/`、`/index`、`/login` 都改为直接返回 `charts.html`。

坑 3：缺少模型权重

现象：

代码块

```
Missing rvm_mobilenetv3.pth
```

处理：把 `rvm_mobilenetv3.pth` 放到 `app.py` 同级目录。

坑 4：Jetson 上 PyTorch CUDA 不可用

现象：

代码块

```
torch.cuda.is_available() == False
```

处理：

- 不要装普通桌面版 PyTorch。
- 按 JetPack 版本安装 NVIDIA Jetson wheel。
- 检查 `LD_LIBRARY_PATH`、CUDA、cuDNN 是否匹配。

坑 5：PyAV 写 H264 失败

现象可能是：

代码块

```
UnknownCodecError: h264
```

或者写视频时报编码器错误。

处理：

代码块

```
sudo apt install -y ffmpeg
pip install av
```

如果仍失败，可以把 `inference_utils.py` 里的编码格式从 `h264` 改成系统支持的编码器，或者改用 OpenCV `VideoWriter`。

坑 6：Femto Mega 和 Astra+ 不完全一样

原项目文档主要写的是 Astra+，但我们复现用的是 Femto Mega。

差异点：

- Femto Mega 常见网络连接场景更多。
- 网络相机可能需要 `create_net_device`。
- Depth / Color profile 和编码格式可能不同。
- 不能盲目套用 USB 相机采集脚本。

坑 7：深度阈值不是毫米值

原预处理脚本使用归一化后的深度图：

```
代码块
1   depth_image = cv2.normalize(...)
2   depth_threshold = 100
```

这意味着阈值会随每帧深度范围变化，不够稳定。

更稳的做法是使用原始深度毫米值做 mask。

11. 推荐最终验证清单 ✅

复现完成后逐项检查：

- ☐ `python3 app.py` 能启动。

- [ ] 浏览器打开根路径直接进入测试页面。
- [ ] 上传视频后终端出现 `POST /upload 200`。
- [ ] 点击生成后终端出现 `GET /process/...`。
- [ ] `uploads/output/` 下出现结果视频。
- [ ] 页面能播放处理后视频。
- [ ] `torch.cuda.is_available()` 为 `True`。
- [ ] `rvm_mobilenetv3.pth` 在 `app.py` 同级目录。
- [ ] 如果使用相机采集，`utils.py` 已补齐。

12. 一句话总结 🎯

这个项目的核心不是“写一个 Flask 页面”，而是把 **Orbbec RGB-D 采集、深度预处理、RVM 视频抠图、Web 调用本地推理** 这几段链路真正接起来。

复现时不要着急一口气跑完整系统。最稳的方法是：

```
代码块
1   先跑相机采集 -> 再跑预处理 -> 再跑模型推理 -> 最后接 Web
```

每一步都能单独验证，项目就会从“看起来很复杂”变成“每一块都能控制”。这才是复现成功的真正姿势。🚀